

EXPENSE TRACKER

Android / Desktop Application · Kivy + Python

Risk Analysis, Security Review Implementation Roadmap & Validation / Testing Plan

Version 2.0 · March 2026



1. Risk Identification

This section identifies all significant risks associated with the Expense Tracker project across technical, security, compliance, usability, and operational dimensions. Each risk is assessed on probability and impact, assigned an overall risk level, and assigned an owner responsible for its mitigation.

1.1 Risk Assessment & Management

Risk levels are calculated from a 5×5 Probability × Impact matrix. The following register covers the current state of the project at version 2.0, after completion of Phases 1 and 2 (Core MVP + Authentication/Roles).

ID	Category	Risk Title	Prob.	Impact	Level	Owner
R-01	Technical	SQLite Concurrency Corruption	Low	Critical	High	Dev Team
R-02	Security	Weak / Stolen Password	Medium	High	High	Dev Team
R-03	Security	SQLite File Direct Access	Medium	High	High	Dev Team
R-04	Technical	App Crash / Data Loss on Mobile	Medium	High	High	QA Team
R-05	Compliance	GDPR / Data Privacy Violation	Low	Critical	High	PM / Legal
R-06	Usability	Poor UX on Small Screens	Medium	Medium	Medium	UX Team
R-07	Technical	Kivy Version Incompatibility	Medium	Medium	Medium	Dev Team
R-08	Technical	FontAwesome TTF Missing on Device	Medium	Low	Low	Dev Team
R-09	Security	Admin Privilege Escalation	Low	Critical	High	Dev Team
R-10	Operations	No Data Backup / Recovery	High	High	Critical	Dev Team
R-11	Technical	Performance Degradation (>10K rows).	Low	Medium	Low	Dev Team
R-12	Compliance	Unencrypted Database at Rest	Medium	High	High	Dev Team

1.2 Mitigation / Preventive Actions

The following table details the specific mitigation or preventive actions assigned to each



identified risk.

ID	Mitigation / Preventive Action
R-01	Use SQLite WAL mode; implement transaction wrappers; add integrity checks on startup.
R-02	Enforce PBKDF2-SHA256 password hashing (260k iterations), enforce min 6-char passwords, add brute-force lockout after 5 failed attempts.
R-03	Store DB in app-private directory; on Android use internal storage (no READ_EXTERNAL_STORAGE permission). Implement SQLCipher encryption for production.
R-04	Wrap all DB calls in try/except; implement auto-save drafts; add crash reporting (Firebase Crashlytics); thorough device testing.
R-05	Add Privacy Policy; allow data export/deletion (Right to Erasure); collect only essential data; no 3rd-party analytics without consent.
R-06	Test on 5-inch and 6-inch screens; use dp() units throughout; provide scrollable layouts for all screens; accessibility testing.
R-07	Pin Kivy to tested version (2.3.x) in requirements.txt; run CI tests on version upgrade; maintain compatibility matrix.
R-08	Bundle fontawesome-webfont.ttf with the APK; implement ASCII fallback already coded; validate font presence at startup.
R-09	Server-side role validation on every admin action; cannot demote last admin; audit log for all role changes; UI-only guard insufficient.
R-10	Implement CSV/JSON export feature; auto-backup to device storage daily; add manual backup/restore UI in Settings; document backup procedure.
R-11	Add SQL indexes on expenses(user_id, date); paginate lists (limit/offset already coded); lazy-load dashboard stats.
R-12	Evaluate SQLCipher for full DB encryption at rest; ensure APK does not expose DB path; rotate encryption key on password change.

1.3 Risk Matrix

The 5×5 matrix below plots each risk by Probability (horizontal) and Impact (vertical). Cell colour indicates overall severity. Risk IDs are placed in the corresponding cell.

Impact \ Prob.	Very Low	Low	Medium	High	Very High
Critical		R-01, R-05, R-09			
High			R-02, R-03, R-04, R-12	R-10	
Medium		R-11	R-06, R-07		



Impact \ Prob.	Very Low	Low	Medium	High	Very High
Low			R-08		
Very Low					

Colour legend: Red = Critical | Orange = High | Amber = Medium | Green = Low



2. Code Security & Data Privacy

2.1 Authentication & Password Security

The application uses PBKDF2-HMAC-SHA256 with a randomly generated 16-byte salt and 260,000 iterations — well above NIST SP 800-132 recommendations. Each password is stored as "salt\$hash" in the users table, never in plain text. The verify step uses `hmac.compare_digest()` to prevent timing attacks.

Planned enhancements: account lockout after 5 consecutive failed login attempts; session token expiry for multi-device support; optional two-factor authentication (TOTP) for admin accounts.

2.2 Role-Based Access Control (RBAC)

Two roles are enforced: user and admin. Every sensitive operation in `logic.py` begins with an `is_admin()` or `is_logged_in()` guard. The database layer additionally enforces ownership via `user_id` foreign keys — even if a UI guard were bypassed, direct DB queries still scope data correctly.

Admin capabilities: view all users, create/delete accounts, promote/demote roles, view global statistics.

User capabilities: manage own expenses and budgets only. Cannot access other users' data.

Last-admin protection: The system prevents demotion or deletion of the sole remaining admin account.

2.3 Database Security

SQLite stores data in a local file (`expenses.db`). `PRAGMA foreign_keys = ON` is enforced on every connection. `CASCADE DELETE` ensures orphaned records are automatically cleaned up when users or categories are removed.

Current gap: the database file is not encrypted at rest. This is acceptable for a development build but must be addressed before production deployment on Android. The recommended solution is `SQLCipher` (an open-source SQLite encryption extension) which provides 256-bit AES encryption transparent to the application code.

2.4 Input Validation & Injection Prevention

All database queries use parameterised statements (`sqlite3 ? placeholders`) — SQL injection is not possible through normal application flow. Input validation in `logic.py` checks: non-empty title, positive numeric amount, valid date format (YYYY-MM-DD), and category selection before any DB write.

The Kivy `TextInput` `input_filter="float"` restricts the amount field to numeric characters at the UI layer, providing an additional defence-in-depth layer.

2.5 Data Privacy & GDPR Considerations

The application collects: username, hashed password, expense records (title, amount, category, date, note), and budget limits. No analytics, advertising identifiers, or biometric data are collected.

- Right to Access: admin can export all data; user data is visible to the account holder only.
- Right to Erasure: `admin_delete_user()` performs a CASCADE DELETE removing all associated expenses and budgets.
- Data Minimisation: the note field is optional; no geolocation, device ID, or usage telemetry is stored.
- Transparency: a Privacy Policy document should be provided before public distribution.
- Data Retention: no automatic retention policy currently exists — recommended to add configurable auto-purge for old records.

2.6 Android Deployment Security

For Android APK distribution: the database must reside in the app's private internal storage (`Context.getFilesDir()`), which is inaccessible to other apps without root access. The `buildozer.spec` should request only `android.permissions = VIBRATE` — no storage or internet permissions are required for core functionality, minimising the attack surface.

The `fontawesome-webfont.ttf` asset must be bundled inside the APK (`source.include_exts` includes `ttf`). It must not be loaded from an external URL to prevent supply-chain attacks.



3. Implementation Roadmap

The project is divided into five phases spanning sixteen weeks. Phases 1 and 2 are complete; Phases 3–5 represent planned work. Each phase has defined deliverables that can be independently verified.

Phase	Timeline	Deliverables / Key Tasks
Phase 1 — Core MVP	Weeks 1–4 (COMPLETED)	• SQLite schema: categories, expenses (user-scoped), budgets • Business logic layer: validation, formatting, CRUD operations • Kivy GUI: Dashboard, Expenses, Add/Edit, Stats, Settings screens • Bottom NavBar with FontAwesome icons and fallback • Month navigation, budget progress bars, recent expenses
Phase 2 — Authentication & Roles	Weeks 5–8 (COMPLETED)	• users table with PBKDF2-SHA256 password hashing • Login / Register screens with error feedback • Role system: user vs admin with RBAC guards • Admin Panel: user management, role toggle, delete, create • NavBar adapts tabs based on role after login • Change password in Settings; last-admin protection
Phase 3 — Security Hardening	Weeks 9–10 (PLANNED)	• SQLCipher integration for DB encryption at rest • Login brute-force lockout (5 attempts → 30-minute cooldown) • Audit log table for admin actions (role changes, user deletions) • Input sanitisation review; XSS-equivalent checks for note fields • Android permissions audit; tighten buildozer.spec
Phase 4 — Data Management & UX	Weeks 11–13 (PLANNED)	• CSV / JSON export and import for expense data • Daily auto-backup to device storage • Pagination / lazy loading for expense lists >100 rows • Category management UI (add/edit/delete) for admins • Dark/light theme toggle; font-size accessibility option

Phase	Timeline	Deliverables / Key Tasks
		• Multi-currency support (currency selector in Settings)
Phase 5 — Polish & Release	Weeks 14–16 (PLANNED)	• Full regression test suite (unit + integration + UI) • Performance benchmarking: 10,000 expense records • Google Play Store listing preparation & signing • Privacy Policy and Terms of Service documents • User documentation / in-app onboarding tooltips • Crash reporting integration (Firebase Crashlytics or Sentry)



4. Validation & Testing Plan

4.1 Testing Strategy Overview

The project follows a multi-layer testing approach: Unit Testing → Integration Testing → System Testing → User Acceptance Testing (UAT). Automated tests cover the logic and database layers; manual exploratory testing covers the Kivy GUI on real devices.

4.2 Unit Testing (logic.py & database.py)

Framework: pytest. Each function in logic.py and database.py has a corresponding test file. Tests run against an in-memory SQLite database (:memory:) to avoid polluting the production database.

Auth tests: register with valid/invalid inputs; login with correct/wrong password; login after wrong password 5× (lockout); hashing produces different hashes for same input; verify_password correct/incorrect.

Expense tests: add/update/delete expense; validate_expense rejects empty title, non-numeric amount, invalid date; ownership checks prevent cross-user access; admin bypasses ownership check.

Budget tests: set budget; retrieve budgets; budget_pct calculation; over_budget flag triggers correctly.

Role tests: is_admin() / is_logged_in() correct for both roles; admin_change_role prevents demoting last admin; admin_delete_user cascade-deletes expenses.

Coverage target: ≥ 85% line coverage on logic.py and database.py.

4.3 Integration Testing

Integration tests verify that logic.py correctly calls database.py and that data flows correctly through both layers end-to-end.

- Full user journey: register → login → add expense → view in list → edit → delete → logout
- Admin journey: login as admin → view all users → create user → change role → delete user
- Budget journey: set budget → add expenses that exceed it → verify over_budget flag
- Month navigation: add expenses across 3 months → verify dashboard shows correct month total
- Data isolation: user A cannot read user B's expenses; admin can read both

4.4 GUI / System Testing

GUI testing is performed manually on desktop (Windows/Linux) and on Android emulator. The following test scenarios are scripted:

TC-ID	Test Case	Steps	Expected Result
TC-01	Login with default admin	Open app → enter admin/Admin123 → tap Sign In	Navigates to Dashboard; NavBar shows Users tab

TC-ID	Test Case	Steps	Expected Result
TC-02	Login with wrong password	Enter admin / wrongpass → Sign In	Error label shows 'Incorrect password'
TC-03	Register new user	Tap Register → fill form → Create Account	Success popup; redirected to Login
TC-04	Add expense	Login → Add tab → fill all fields → Save	Expense appears in Expenses list
TC-05	Edit expense	Expenses tab → tap Edit on any row → change amount → Save	Updated amount shown in list
TC-06	Delete expense	Expenses tab → tap Del → confirm	Expense removed from list and DB
TC-07	Budget warning	Settings → set \$10 budget for Food → add \$15 food expense	Red progress bar and 'Over budget' label on Dashboard
TC-08	Month navigation	Dashboard → tap ← arrow	Month label updates; totals recalculate for previous month
TC-09	Admin user management	Admin → Users tab → change role → delete user	Role badge updates; deleted user no longer listed
TC-10	Change password	Settings → Change Password → enter correct old pw + new pw	Success popup; old password no longer works
TC-11	Logout	Dashboard → tap Logout	Returns to Login screen; current_user is None
TC-12	FontAwesome fallback	Rename TTF file → restart app	App loads with ASCII fallbacks; no crash

4.5 Performance Testing

Load test: Insert 10,000 expense records via script; measure Dashboard load time (target: < 1 second); measure Expenses list scroll FPS (target: 60 fps).

Memory test: Monitor RSS memory usage on Android with 1,000 and 10,000 records; target < 150 MB.

Stress test: Simulate 50 rapid add/delete operations; verify database integrity with PRAGMA integrity_check after each run.

Battery test: Run app in background for 30 minutes; verify battery drain < 2% (no background tasks expected).

4.6 Security Testing

- SQL Injection: attempt to inject SQL via all TextInput fields (e.g., title = ""; DROP

TABLE expenses;--"). Verify parameterised queries prevent execution.

- Role bypass: call `logic.admin_get_all_users()` while logged in as a regular user. Verify empty list returned.
- Ownership bypass: attempt to delete another user's expense by direct ID manipulation. Verify denied.
- Password hash verification: read DB directly; confirm no plaintext passwords; confirm salts are unique per user.
- Brute force (planned): after 5 failed logins, verify lockout message and 30-minute cooldown enforced.

4.7 Validation Methods

Black-box testing: Test all screens against acceptance criteria without knowledge of internal implementation. Performed by a tester unfamiliar with the codebase.

White-box testing: Review `logic.py` branches for coverage; ensure every guard clause (`is_admin`, `is_logged_in`, ownership check) is tested with both a passing and failing case.

Regression testing: After every code change, run the full pytest suite (automated) and re-run critical manual test cases TC-01 through TC-11.

Boundary testing: Amount = 0 (should fail), amount = 0.01 (should pass), amount = 999999.99 (should pass); date = invalid string (should fail); username = 2 chars (should fail), 3 chars (should pass).

Cross-platform testing: Run on Windows 10, Ubuntu 22.04, and Android 12 (emulator). Verify identical behaviour and identical DB schema creation.



5. Success Criteria

A project is considered successful when ALL criteria in the following table are met and verified. Criteria are grouped by quality dimension. Each criterion must be demonstrably satisfied — either by an automated test result, a manual test walkthrough, or a documented review finding.

Category	Success Criteria
Functional	All 12 manual test cases (TC-01 to TC-12) pass on desktop and Android emulator.
Functional	User can complete the full expense lifecycle (add → edit → delete) without errors.
Functional	Admin can manage all users: create, change role, delete, with correct data isolation.
Functional	Budget limits are correctly enforced and visualised on the Dashboard.
Functional	Month navigation displays accurate totals for all months with data.
Security	No plaintext passwords in the database at any point.
Security	SQL injection attempts via all input fields produce no unintended DB changes.
Security	A regular user cannot access or modify another user's data through any code path.
Security	Admin-only functions return an error when called by a non-admin session.
Security	PBKDF2 password hash verifies correctly after app restart (session persists).
Performance	Dashboard loads within 1 second for a dataset of 1,000 expenses.
Performance	Expense list scrolls without visible jank (≥ 30 fps) for 500+ rows.
Performance	App startup time < 3 seconds on mid-range Android device (2 GB RAM).
Performance	Memory usage stays below 150 MB during normal usage.
Code Quality	pytest unit test coverage $\geq 85\%$ on logic.py and database.py.
Code Quality	No SyntaxError or ImportError on Python 3.10–3.13.
Code Quality	All DB queries use parameterised statements (no f-string SQL).
Code Quality	Zero critical issues identified in a manual code security review.
Usability	A new user can register, log in, and add their first expense within 2 minutes without instructions.
Usability	All interactive elements (buttons, spinners) respond to touch on 5-inch



Category	Success Criteria
Usability	screen.
	• Error messages are descriptive and actionable (not generic 'Error occurred').
Usability	• App functions correctly in both portrait mode on mobile and windowed mode on desktop.
Maintainability	• 3-layer architecture (database / logic / gui) is clearly separated; no DB calls from gui.py.
Maintainability	• PlantUML diagram accurately reflects the codebase and passes syntax validation.
Maintainability	• README.md describes setup, default credentials, and build instructions.
Maintainability	• All risk mitigations from the Risk Register are tracked in the project backlog.

